

Chapter 5 – Engineering Sprint

Building Complete Business Transactions with SOQL, DML and Apex

Hospital OPD Management System

PART 1: Introduction

1.1 Aim

To implement a complete Hospital OPD business transaction using Salesforce Apex, SOQL, and DML for managing patient appointments efficiently.

1.2 Objective

- Retrieve patient information using SOQL.
- Retrieve doctor information using SOQL.
- Prevent duplicate appointments.
- Create appointment records using DML.
- Update appointment status.
- Build a complete Hospital OPD business transaction.

1.3 Project Description

The Hospital OPD Management System is developed using Salesforce to manage patient appointments. The system retrieves patient and doctor information, validates appointment requests, prevents duplicate appointments, creates appointment records, updates appointment status, and completes the business transaction using Apex, SOQL, and DML.

PART 2: Technologies and Objects Used

2.1 Technologies Used

- Salesforce CRM
- Apex
- SOQL
- DML
- Salesforce CLI

- Visual Studio Code
- Trailhead Playground

2.2 Objects Used

Object	Purpose
Patient	Stores patient information
Doctor	Stores doctor details
Appointment	Stores appointment details
Prescription	Stores prescription details

PART 3: Sprint 7 – Retrieve Patient Information

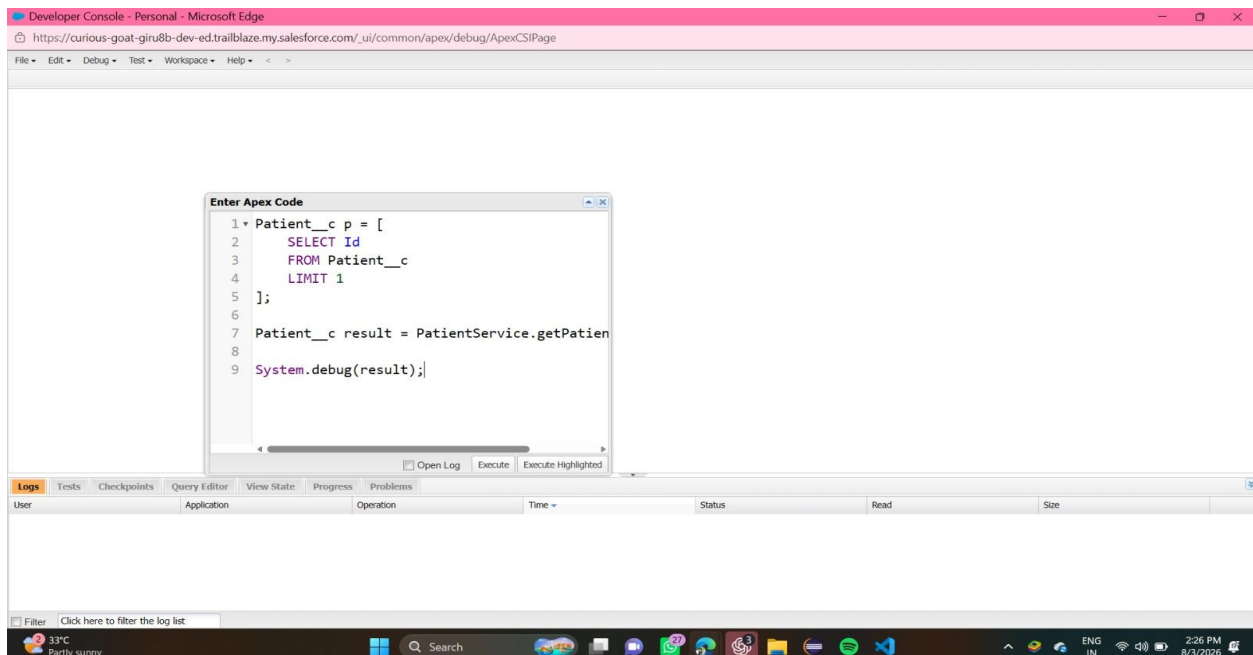
Objective

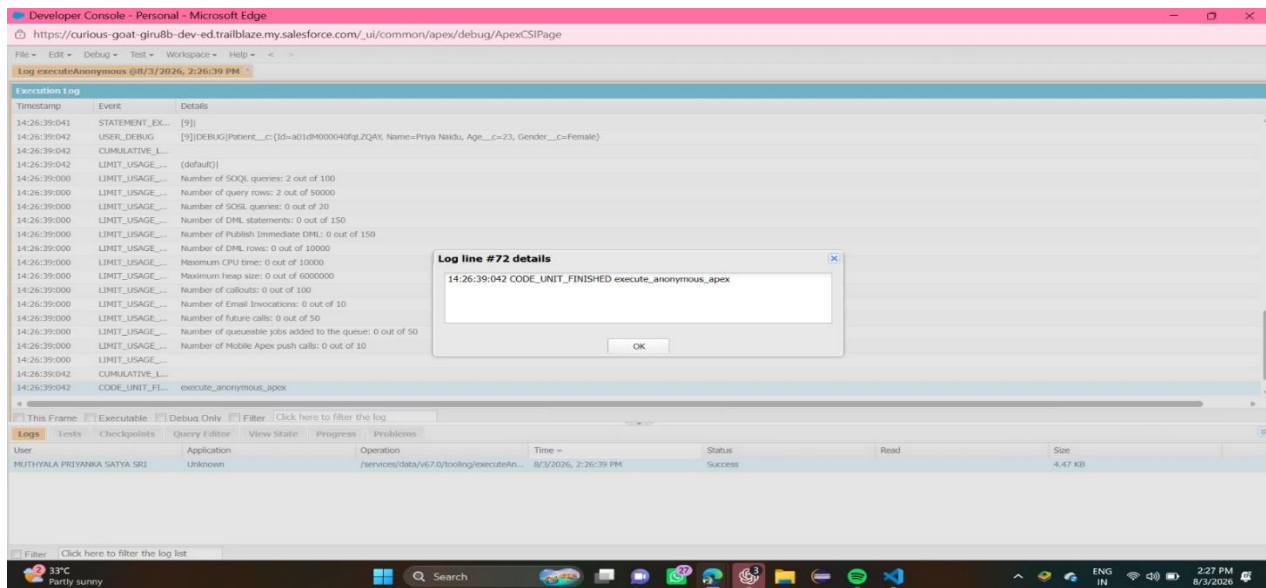
Retrieve patient information before processing an appointment.

Implementation

Created the **PatientService** Apex class to retrieve patient details using SOQL. Only the required fields were queried to improve efficiency.

Output





Sprint 8 – Retrieve Doctor Information

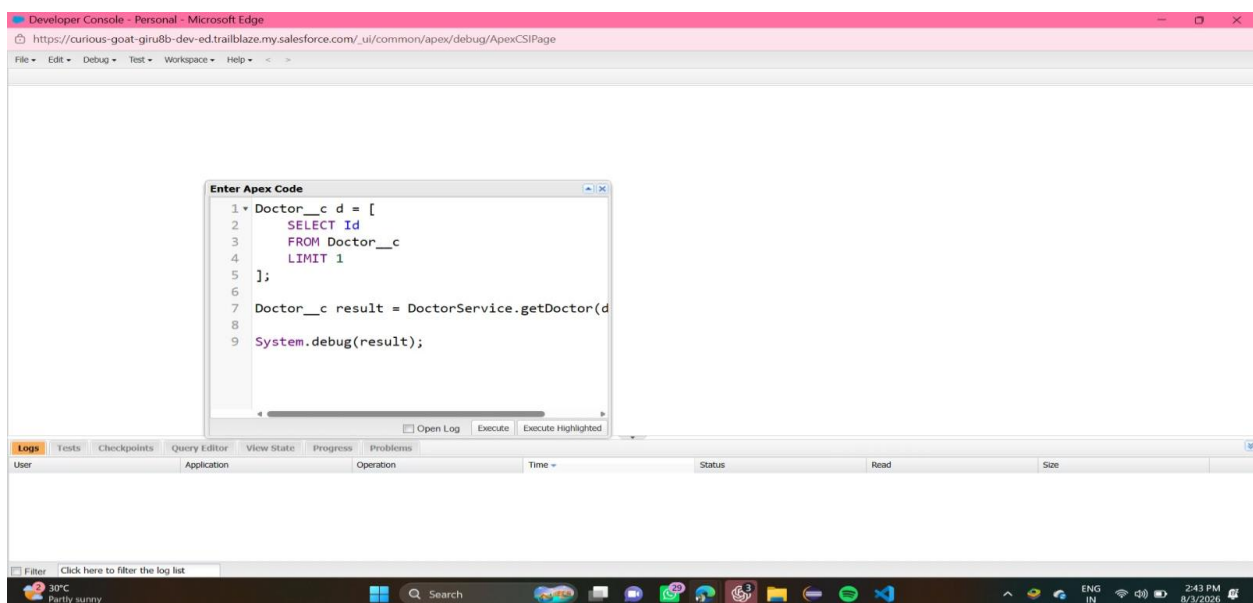
Objective

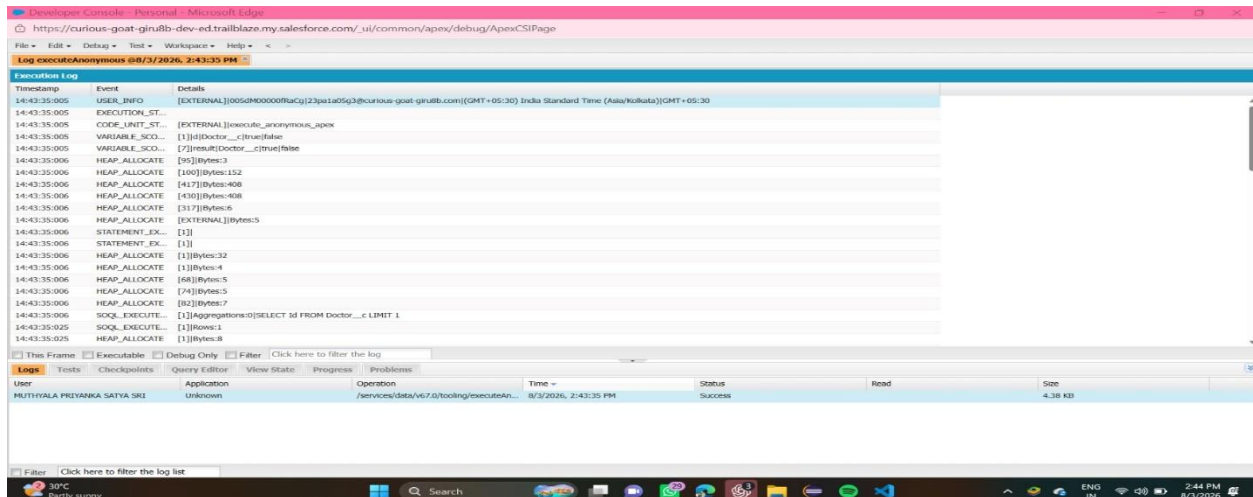
Retrieve doctor information required for appointment validation.

Implementation

Created the **DoctorService** Apex class to retrieve doctor details using SOQL before processing appointments.

Output:





Sprint 9 – Prevent Duplicate Appointments

Objective

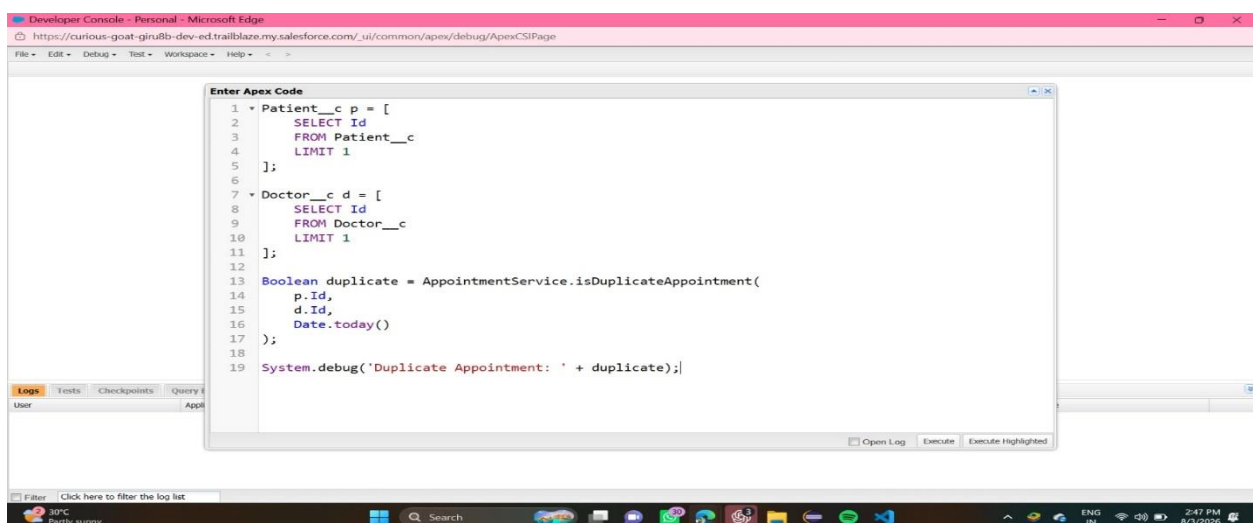
Prevent duplicate appointment bookings for the same patient, doctor, and appointment date.

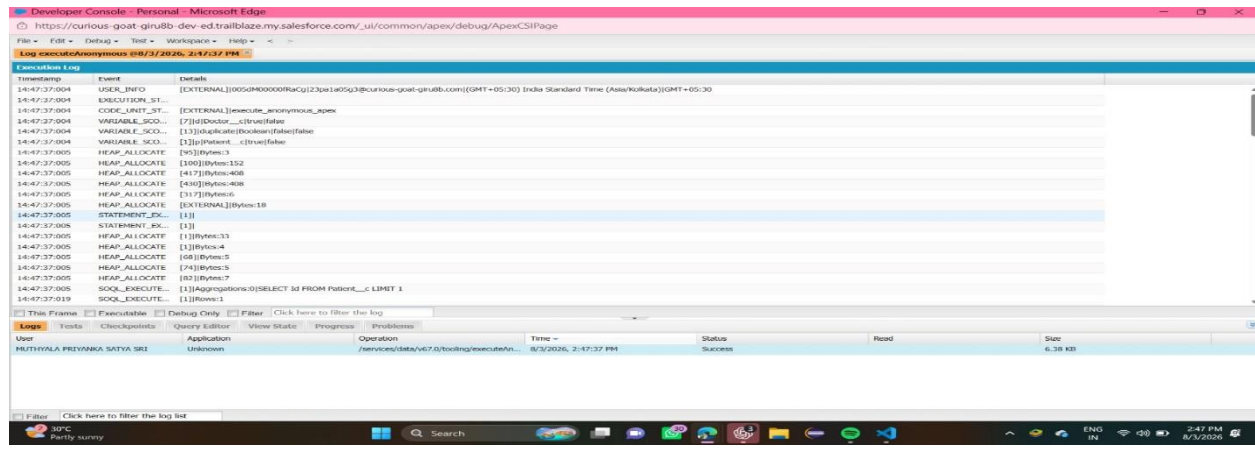
Implementation

Implemented duplicate validation using:

- AppointmentTrigger
- AppointmentTriggerHandler
- DuplicateAppointmentService

The system checks existing appointments before allowing a new appointment to be created.





Sprint 10 – Create Appointment

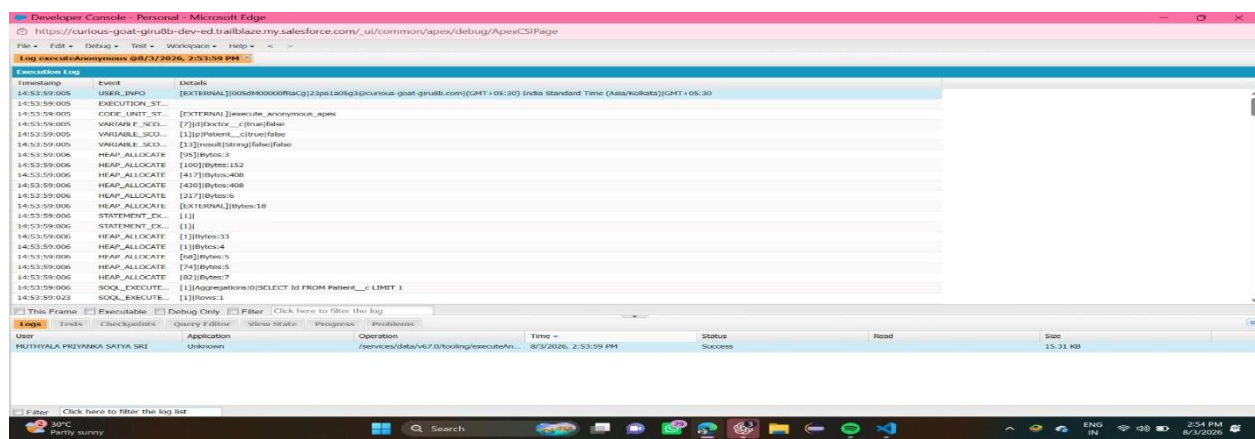
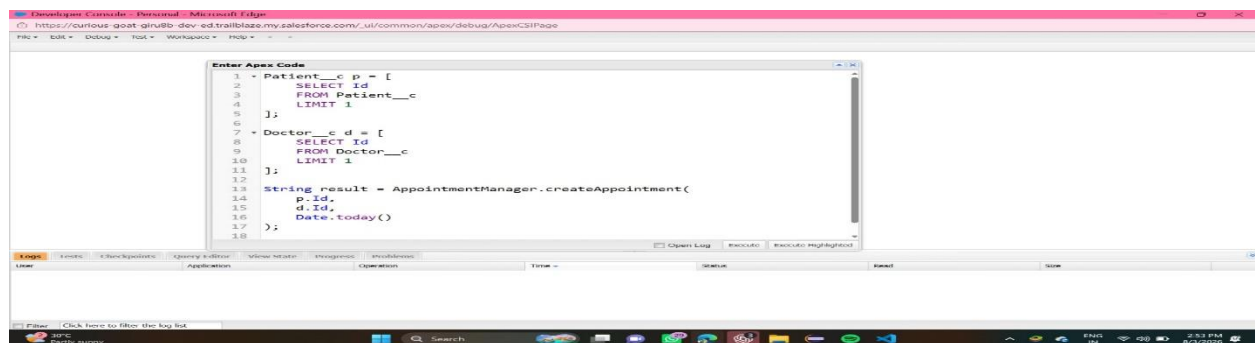
Objective

Create a new appointment after successful validation.

Implementation

Implemented the **CreateAppointmentService** class to create appointment records using DML Insert after all validations were completed.

Output:



PART 7: Sprint 11 – Update Appointment Status

Objective

Update the appointment status after consultation.

Implementation

Implemented the **UpdateAppointmentService** class to update the appointment status using DML Update.

Output:

The screenshot displays the Salesforce Developer Console interface. The top section shows the 'Enter Apex Code' window with the following code:

```
1 Appointment_Date__c appt = [  
2   SELECT Id  
3   FROM Appointment_Date__c  
4   WHERE Status__c = 'Scheduled'  
5   LIMIT 1  
6 ];  
7  
8 String result = UpdateAppointmentService.up  
9  
10 System.debug(result);
```

The bottom section shows the 'Execution Log' with the following data:

Timestamp	Event	Details
15:03:58:001	USER_INFO	[EXTERNAL]005d400000f0aCg123pa1a05g3@curious-goat-giru8b.com [GMT+05:30] [India Standard Time (Asia/Kolkata)] GMT+05:30
15:03:58:001	EXECUTION_ST...	
15:03:58:001	CODE_UNIT_ST...	[EXTERNAL]execute_anonymous_apex
15:03:58:001	VARIABLE_SCO...	[1] appt Appointment_Date__c true false
15:03:58:001	VARIABLE_SCO...	[7] result String false false
15:03:58:001	HEAP_ALLOCATE	[95] Bytes:3
15:03:58:001	HEAP_ALLOCATE	[100] Bytes:152
15:03:58:001	HEAP_ALLOCATE	[417] Bytes:408
15:03:58:001	HEAP_ALLOCATE	[430] Bytes:408
15:03:58:001	HEAP_ALLOCATE	[317] Bytes:6
15:03:58:001	HEAP_ALLOCATE	[EXTERNAL] Bytes:5
15:03:58:001	STATEMENT_EX...	[1]
15:03:58:001	STATEMENT_EX...	[1]
15:03:58:001	HEAP_ALLOCATE	[1] Bytes:42
15:03:58:001	HEAP_ALLOCATE	[1] Bytes:4
15:03:58:001	HEAP_ALLOCATE	[68] Bytes:5
15:03:58:001	HEAP_ALLOCATE	[74] Bytes:5
15:03:58:001	HEAP_ALLOCATE	[82] Bytes:7
15:03:58:002	SOQL_EXECUTE...	[1] Aggregations:0 SELECT Id FROM Appointment_Date__c LIMIT 1
15:03:58:022	SOQL_EXECUTE...	[1] Rows:1
15:03:58:022	HEAP_ALLOCATE	[1] Bytes:8

The bottom table shows the execution results:

User	Application	Operation	Time	Status	Read	Size
MUTHYALA PRIYANKA SATYA SRI	Unknown	/services/data/v67.0/tooling/executeAn...	8/3/2026, 3:03:58 PM	Success		13.32 KB
MUTHYALA PRIYANKA SATYA SRI	Unknown	/services/data/v67.0/tooling/executeAn...	8/3/2026, 3:03:19 PM	List has no rows for assignment to SO...	Unread	2.9 KB

PART 8: Sprint 12 – Complete Business Transaction

Objective

Combine all services into a complete Hospital OPD business transaction.

Business Flow

Receive Appointment Request



Retrieve Patient Details



Retrieve Doctor Details



Validate Appointment



Check Duplicate Appointment



Create Appointment



Update Appointment Status

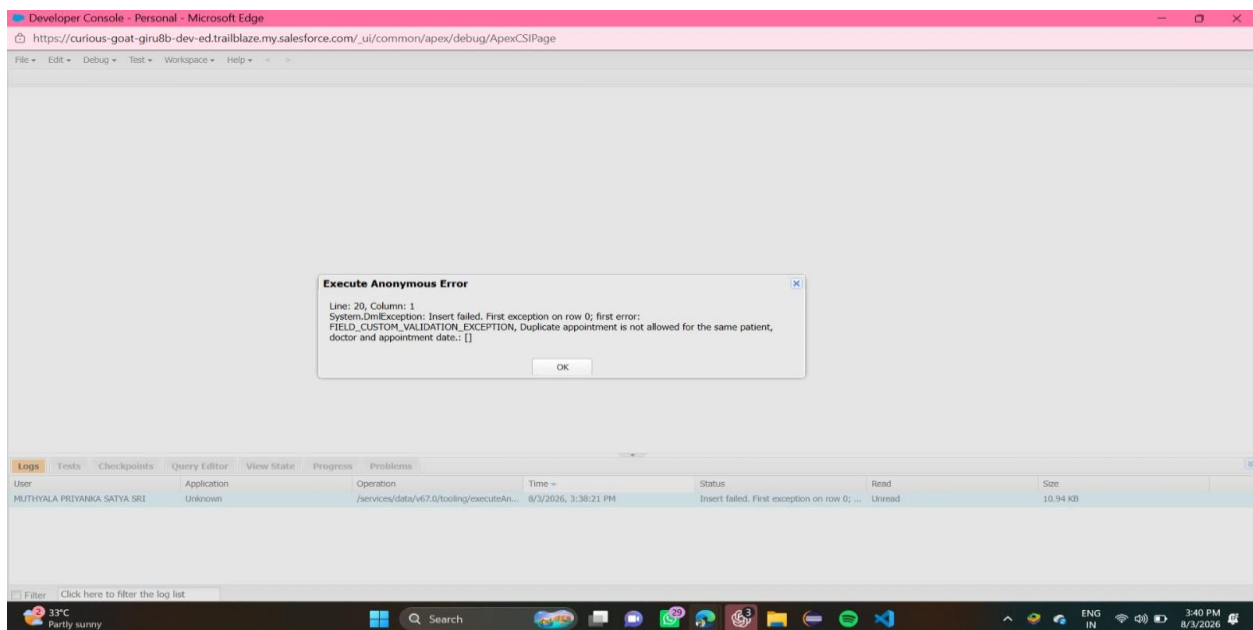
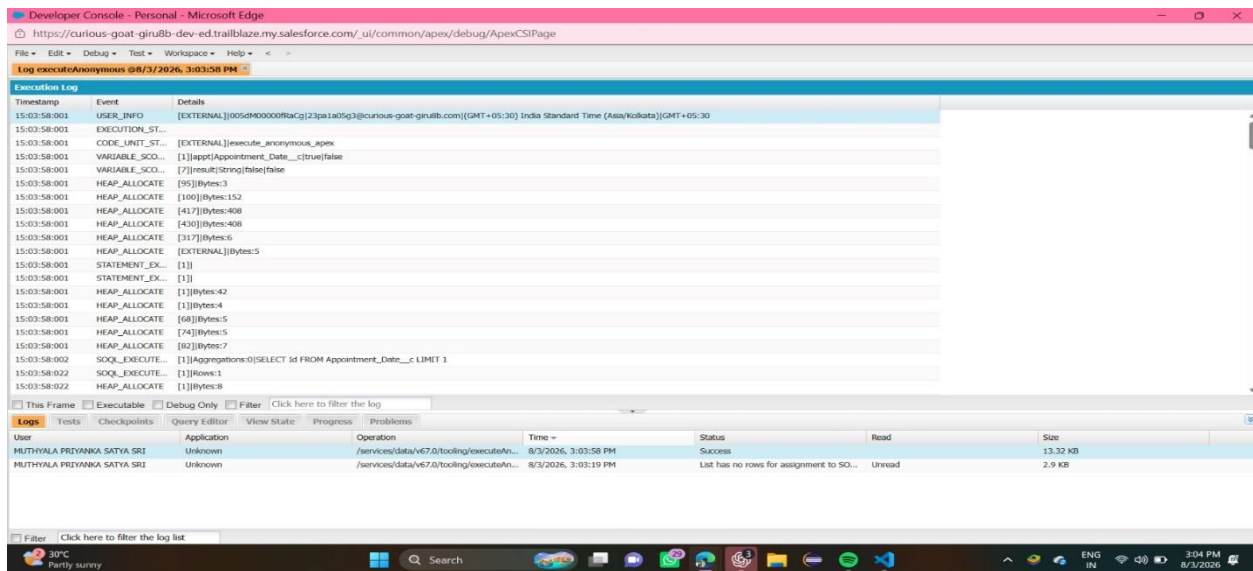


Display Confirmation

Implementation

Developed the **HospitalTransactionService** class to coordinate all business operations, ensuring data retrieval, validation, record creation, and status updates are executed in the correct sequence.

Output:



PART 9: Debug This

Problem 1

Repeated SOQL queries reduce maintainability.

Solution

Use reusable service classes for common queries.

Problem 2

Performing DML before validation may create incorrect records.

Solution

Validate all business rules before performing DML.

Problem 3

Retrieving unnecessary fields wastes resources.

Solution

Query only the required fields.

Problem 4

Placing all logic inside one method reduces maintainability.

Solution

Separate responsibilities into Trigger, Handler, and Service classes.

Conclusion: The Hospital OPD Management System successfully demonstrates the use of Apex, SOQL, and DML to implement complete business transactions. The project retrieves relevant information, validates business rules, prevents duplicate appointments, creates and updates records responsibly, and follows a clean service-based architecture, making the application efficient, maintainable, and aligned with enterprise Salesforce development practices.